

Convolutional Neural Network

Qian Wang

School of Biomedical Engineering

ShanghaiTech University

From MLP to CNN

- In MLP, we flatten all features per sample into a one-dimensional vector.
- Advantages of MLP:
 - Versatility: MLP can be used for a wide range of tasks, including classification, regression, and even unsupervised learning tasks.
 - Universal Approximation: MLPs can approximate any continuous function given enough neurons in the hidden layers.
 - End-to-End Learning: MLPs can learn features directly from raw data without the need for manual feature engineering.

From MLP to CNN

- Disadvantages of MLPs
 - Scalability: MLPs can become computationally expensive and inefficient as the dimensionality of the input data increases.
 - Feature Extraction: MLPs do not inherently understand the spatial or temporal structure of the data, making them less effective for some tasks involving images or time series.
 - Overfitting: With high-dimensional data, MLPs are prone to overfitting, especially if the dataset is not large enough.

Object Detection in Image

- How to detect a specific object from the image?
 - We should have a detector;
 - Sweep the image with the detector.
 - Everywhere in the image uses the same detector.
- Recall what we have done with Viola-Jones detector.



Motivation for CNNs

- **Parameter Sharing:** CNNs use convolutional layers where the same set of weights (kernels, filters, etc.) are applied across different parts of the input, reducing the number of parameters and making the model more efficient.
- **Local Connectivity:** CNNs exploit the local correlation of the input data by focusing on local regions (receptive fields) of the input, which is particularly useful for image and video processing.
- **Hierarchical Feature Learning:** CNNs automatically learn hierarchical features from the data. Lower-level layers learn simple features (edges, textures), while higher-level layers learn more complex features (shapes, objects).
- **Translation Invariance:** Due to the convolutional operation, CNNs are more robust to translations of the input data, which is essential for tasks like image recognition.

Convolution

- To satisfy with the previous principles, one may define the following computation rule:

$$[\mathbf{H}]_{i,j} = u + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} [\mathbf{V}]_{a,b} [\mathbf{X}]_{i+a,j+b}.$$

- $\mathbf{X}$ – input image, $\mathbf{V}$ – kernel, $\mathbf{H}$ – feature map;
- i, j – spatial coordinate;
- a, b – offset in local neighborhood Δ .

- Example:

- Input: 3×3 image;
- Kernel: 2×2.

Input		Kernel		Output																	
<table border="1"><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table>	0	1	2	3	4	5	6	7	8	*	<table border="1"><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table border="1"><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

Convolution

- In mathematics, a real convolution is defined with kernel flipping.
 - Correlation vs. convolution.

$$(f * g)(\mathbf{x}) = \int f(\mathbf{z})g(\mathbf{x} - \mathbf{z})d\mathbf{z}.$$

$$(f * g)(i) = \sum_a f(a)g(i - a).$$

$$(f * g)(i, j) = \sum_a \sum_b f(a, b)g(i - a, j - b).$$

- We often neglect this issue in deep learning, as the kernels are learnable.

Convolution for Images

- The kernel has many parameters, which are **learnable**.
 - And it is exactly the task of learning.
- In general, with the input size $n_h \times n_w$ and the kernel size $k_h \times k_w$, the output size is $(n_h - k_h + 1) \times (n_w - k_w + 1)$.
 - We will be able to keep the size unchanged between input and output, by padding zeros along the boundary of the image.

Input		Kernel		Output																	
<table border="1"><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table>	0	1	2	3	4	5	6	7	8	*	<table border="1"><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table border="1"><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

Receptive Field

- A kernel has a limited receptive field to cover the input image.
 - Example: 2×2 on the right.

Input		Kernel		Output																	
<table border="1"><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table>	0	1	2	3	4	5	6	7	8	*	<table border="1"><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table border="1"><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

- Sometimes we need a larger receptive field to support our computation.
 - How should we do?

Padding

- Convolution may reduce data size.
 - With the input size n and the kernel size k (e.g., assume the same width and height for an input image), the output size is $n - k + 1$.
- In many scenarios, we hope the input and output are exactly matched in their sizes.
 - Padding can help solve this problem.

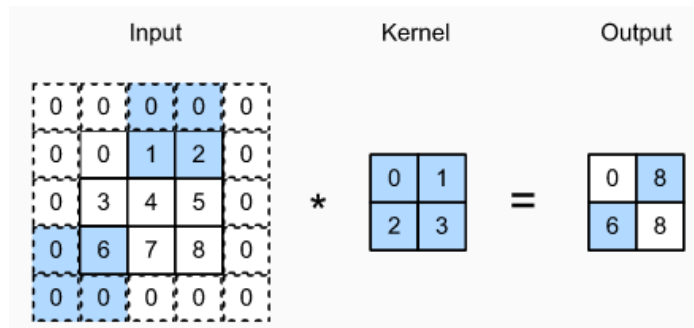
Padding

- With the input size n and the kernel size k , if we add a total of p rows of padding (usually half on top and half on bottom) and p columns (half on the left and half on the right), the output size is $n - k + p + 1$.

Input		Kernel		Output																																													
<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>2</td><td>0</td></tr><tr><td>0</td><td>3</td><td>4</td><td>5</td><td>0</td></tr><tr><td>0</td><td>6</td><td>7</td><td>8</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	0	0	0	1	2	0	0	3	4	5	0	0	6	7	8	0	0	0	0	0	0	*	<table><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table><tr><td>0</td><td>3</td><td>8</td><td>4</td></tr><tr><td>9</td><td>19</td><td>25</td><td>10</td></tr><tr><td>21</td><td>37</td><td>43</td><td>16</td></tr><tr><td>6</td><td>7</td><td>8</td><td>0</td></tr></table>	0	3	8	4	9	19	25	10	21	37	43	16	6	7	8	0
0	0	0	0	0																																													
0	0	1	2	0																																													
0	3	4	5	0																																													
0	6	7	8	0																																													
0	0	0	0	0																																													
0	1																																																
2	3																																																
0	3	8	4																																														
9	19	25	10																																														
21	37	43	16																																														
6	7	8	0																																														

Stride

- In previous examples, we default to sliding one element at a time.
- However, sometimes, either for computational efficiency or because we wish to downsample, we move our window more than one element at a time, skipping the intermediate locations.
- We refer to the number of rows and columns traversed per slide as the **stride**.

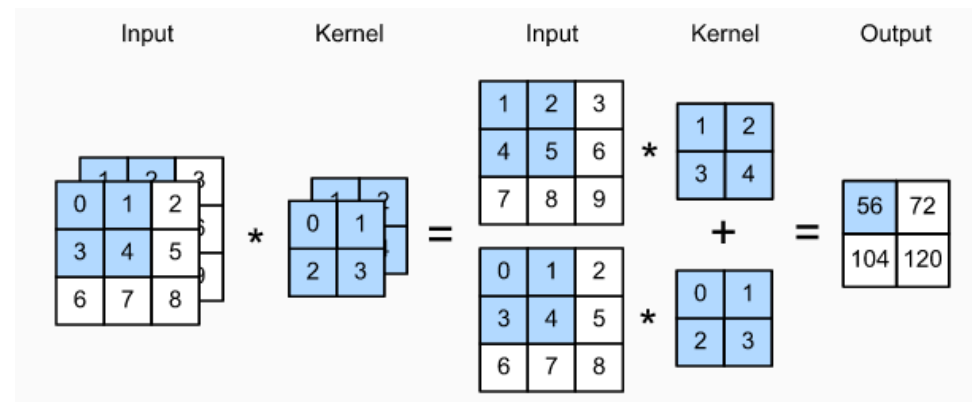


Summary: Padding and Stride

- Padding and stride can be used to adjust the dimensionality of the data effectively.
- Padding can increase the height and width of the output.
 - This is often used to give the output the same height and width as the input.
- Stride can reduce the resolution of the output.
 - For example, reduce the height and width of the output to only $1/s$ of the height and width of the input (s is an integer greater than 1).
- The output size is $(n - k + p)/s + 1$.

Channel

- When the input data contain multiple channels, we need to construct a convolution kernel, whose number of input channels is the same with the number of channels of the input data.
 - Only in this way the kernel can perform cross-correlation with the input data.

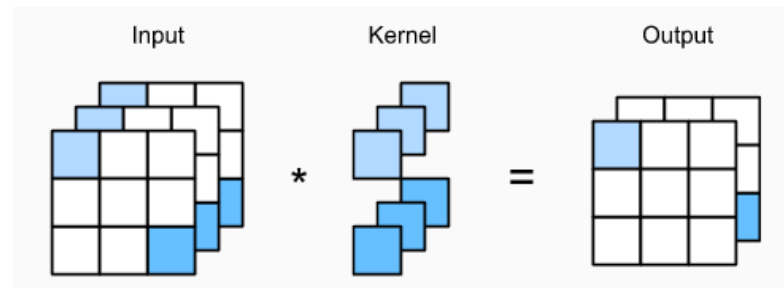


Channel

- Regardless of the number of input channels, so far we always ended up with one output channel.
- It turns out to be essential to have multiple channels at each layer.
 - Each channel is responding to some different set of features.
 - The result of each output channel is calculated from the convolution kernel corresponding to that channel.
 - The kernel takes input from all channels in the input.
- Questions (for kernels):
 - How to determine the number of input channels?
 - How to determine the number of output channels?

1×1 Convolution

- 1×1 convolution is special as it has the minimum receptive field. The only computation occurs on the channel dimension.



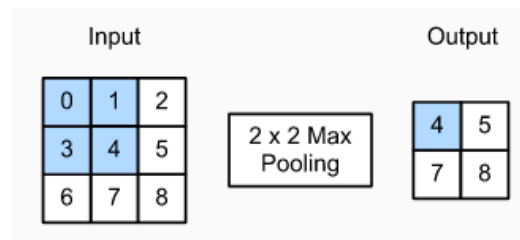
- The 1×1 convolutional layer is equivalent to the fully-connected layer, when applied on a per pixel basis.
- It is typically used to adjust the number of channels between network layers and to control model complexity.

Pooling

- Often, as we process images, we want to gradually reduce the spatial resolution of our hidden representations.
 - Aggregate information so that the higher up we go in the network, the larger the receptive field (in the input) to which each hidden node is sensitive.
 - Moreover, when detecting lower-level features, such as edges, we often want our representations to be somewhat invariant to translation.
 - Pooling serves the dual purposes of mitigating the sensitivity of convolutional layers to location and of spatially downsampling representations.

Pooling

- Pooling operator consists of a fixed-shape window that slides in the input according to its stride, computing a single output for each location traversed by window.
 - Pooling operators are deterministic, typically calculating either the maximum or the average value of the elements in the pooling window.
 - Similar to convolutional layer, pooling can be perceived as kernels.
 - Unlike the convolutional layer, pooling contains no parameters (there is no learnable kernel).
 - Multiple channels are usually processed by pooling operators independently.



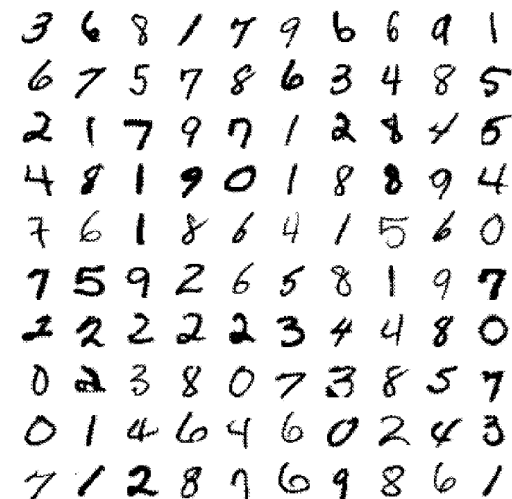
LeNet

- Now we have all ingredients to assemble a real CNN.
- In 1998, LeNet was proposed for the purpose of recognizing handwritten digits in MNIST images.

Gradient-Based Learning Applied to Document Recognition

YANN LECUN, MEMBER, IEEE, LÉON BOTTOU, YOSHUA BENGIO, AND PATRICK HAFFNER

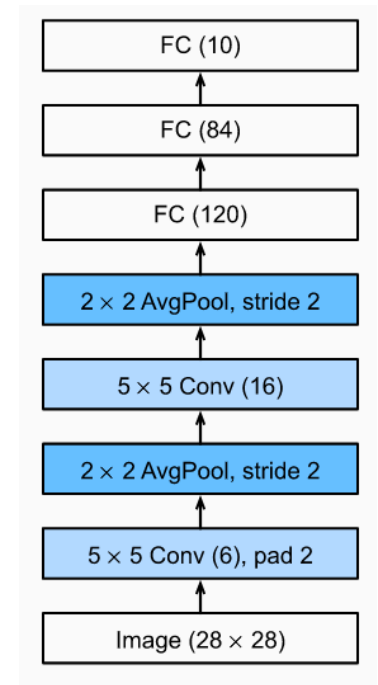
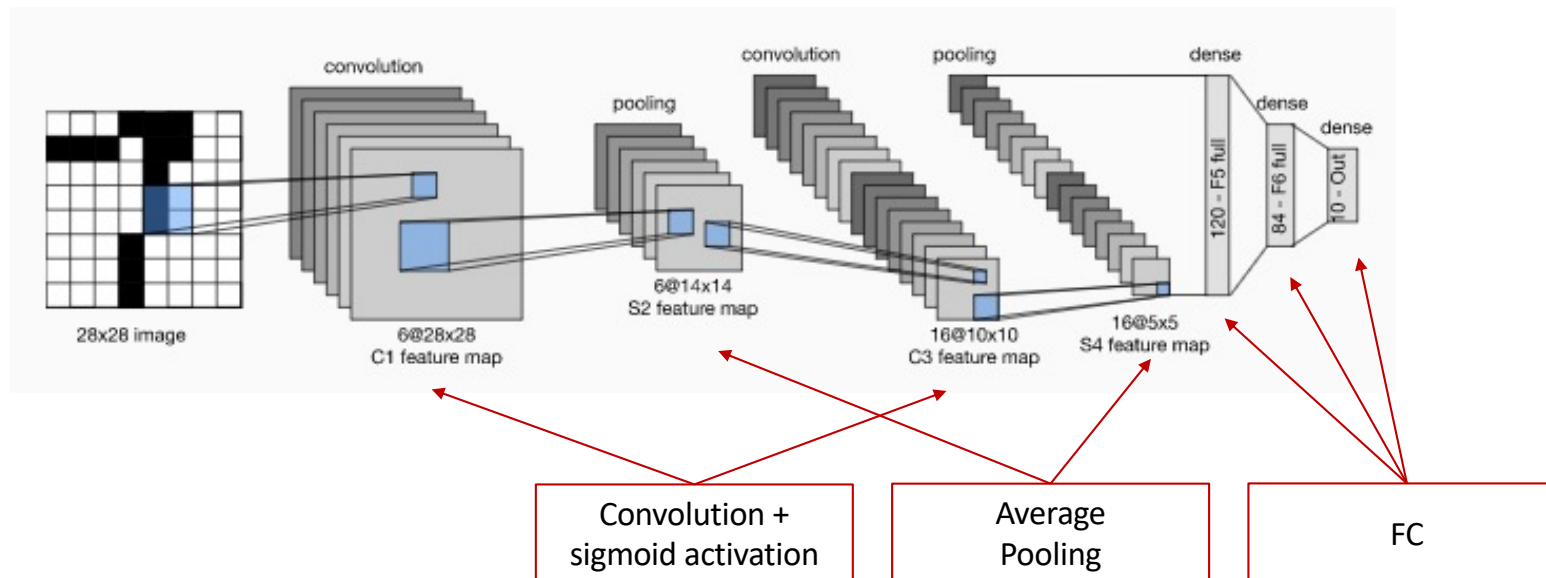
Multilayer neural networks trained with the back-propagation algorithm constitute the best example of a successful gradient-based learning technique. Given an appropriate network architecture, gradient-based learning algorithms can be used to synthesize a complex decision surface that can classify high-dimensional patterns, such as handwritten characters, with minimal preprocessing. This paper reviews various methods applied to handwritten character recognition and compares them on a standard handwritten digit recognition task. Convolutional neural networks, which are specifically designed to deal with the variability of two dimensional (2-D) shapes, are shown to outperform all other techniques.



<https://ieeexplore.ieee.org/document/726791>

LeNet

- Architecture:
 - A convolutional encoder consisting of two convolutional layers;
 - A block consisting of three fully-connected layers.



End-to-end Learning Paradigm

- Conventional machine learning:
 - Obtain an interesting dataset;
 - Preprocess the dataset with hand-crafted features;
 - Dump to your favorite classifier.
- CNN provides an **end-to-end** way for representation learning.

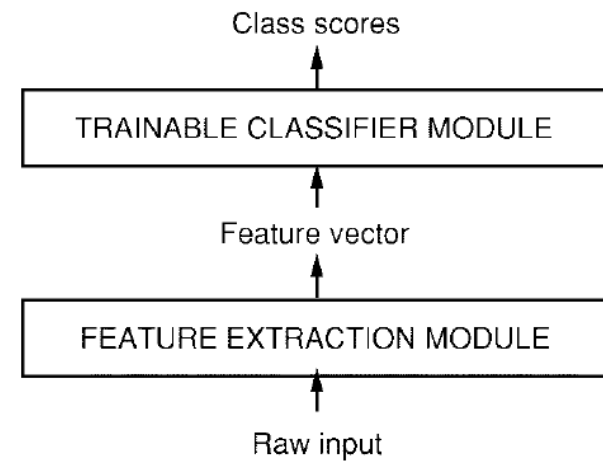


Fig. 1. Traditional pattern recognition is performed with two modules: a fixed feature extractor and a trainable classifier.

AlexNet

- LeNet didn't really draw much attention, until its re-invention in 2012.
- This time, it was AlexNet.

ImageNet Classification with Deep Convolutional Neural Networks

Part of: [Advances in Neural Information Processing Systems 25 \(NIPS 2012\)](#)

[\[PDF\]](#) [\[BibTeX\]](#) [\[Supplemental\]](#)

Authors

- [Alex Krizhevsky](#)
- [Ilya Sutskever](#)
- [Geoffrey E. Hinton](#)

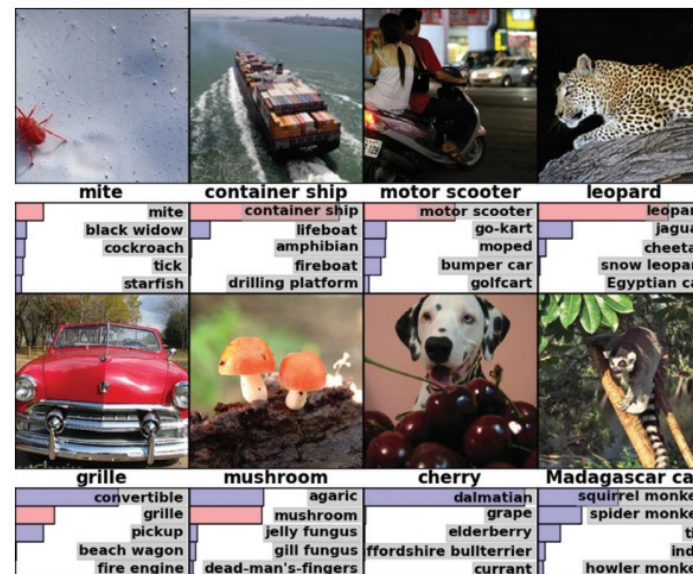


Geoffrey E. Hinton

ImageNet and ILSVRC

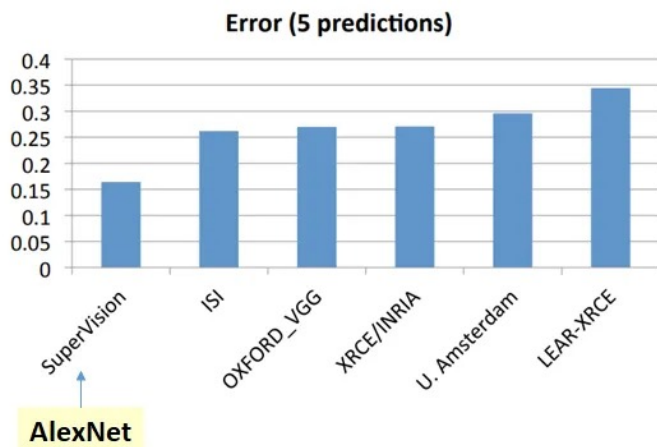
- ImageNet is a large-scale visual database, which spans 15 million images across 22,000 categories.
- Large-Scale Visual Recognition Challenge (ILSVRC) is an annual competition that was held in 2010-2017.
 - It leverages a subset of the ImageNet dataset, focusing on approximately 1,000 categories with over 1.2 million training images, 50,000 validation images, and 100,000 test images.
 - The primary task is image classification, from which top-1 and top-5 error rates can be computed (top-5 error rate would be the percent of images where the correct label is not one of the model's five most likely labels).

IMAGENET



AlexNet

- In 2012, AlexNet, a groundbreaking CNN architecture, introduced significant advancements in image classification, leading to a major breakthrough in ILSVRC.



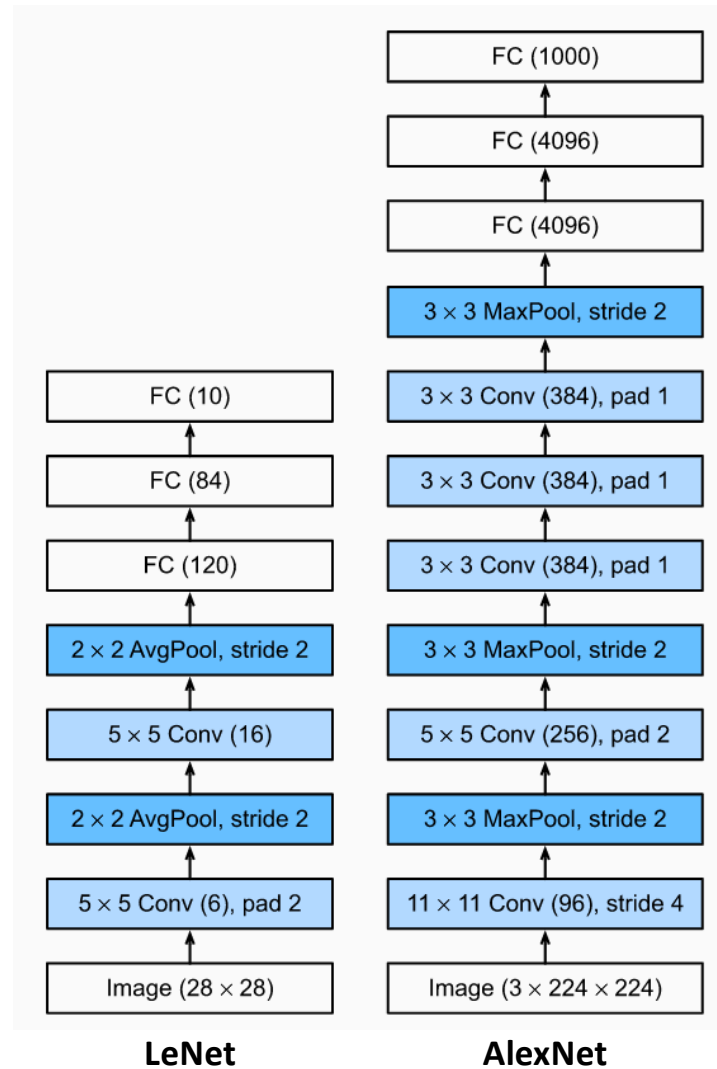
Classification results of ILSVRC 2012



Best results for more years

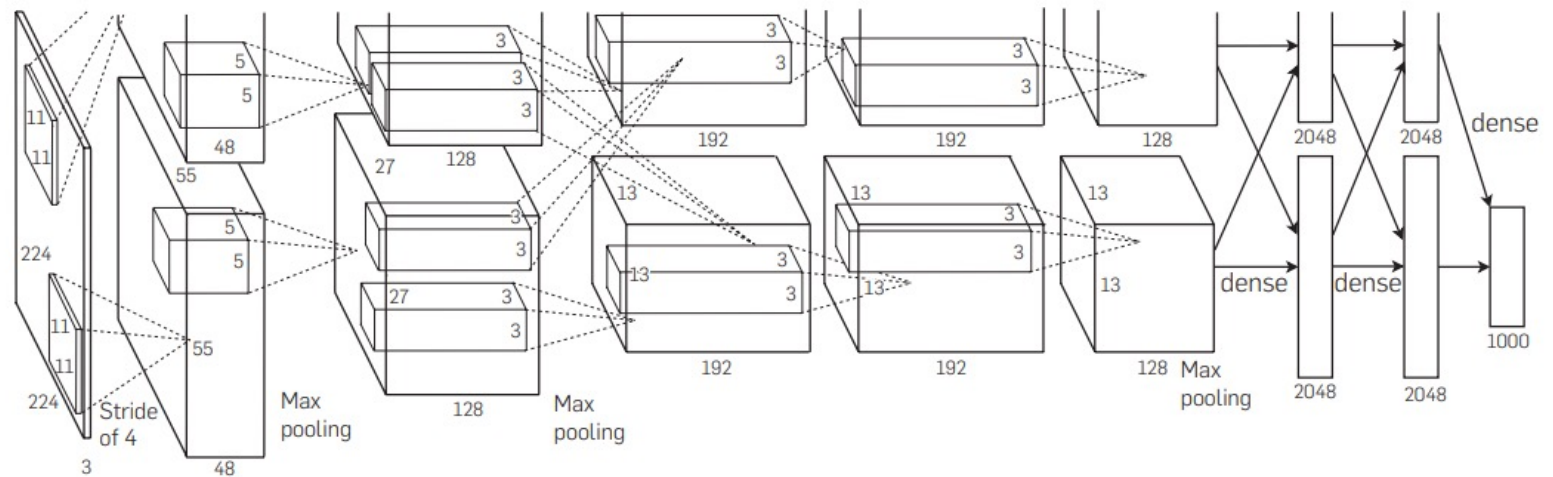
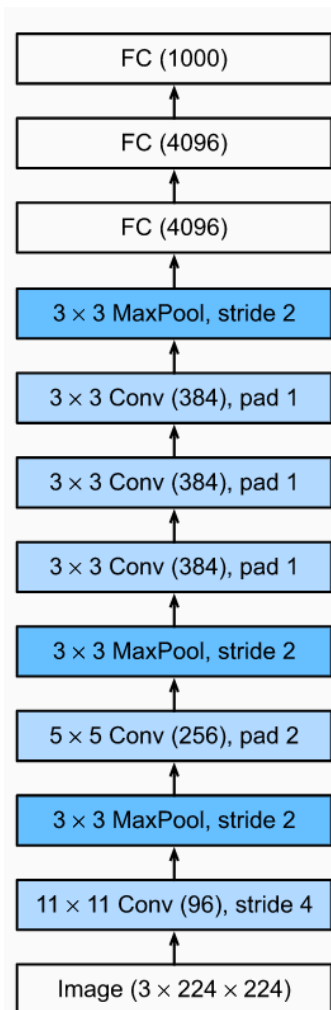
<https://www.image-net.org/challenges/LSVRC/2012/analysis/>

- The structures of LeNet and AlexNet are very similar in first look, yet they differ a lot in fact.



- Network complexity
- ReLU activation function
- Dropout
- Image augmentation
- Utilization of GPUs

https://www.image-net.org/static_files/files/supervision.pdf



The devil is in the details!

- How to calculate the input/output sizes of individual layers?
- Why are there lower and upper data flows?